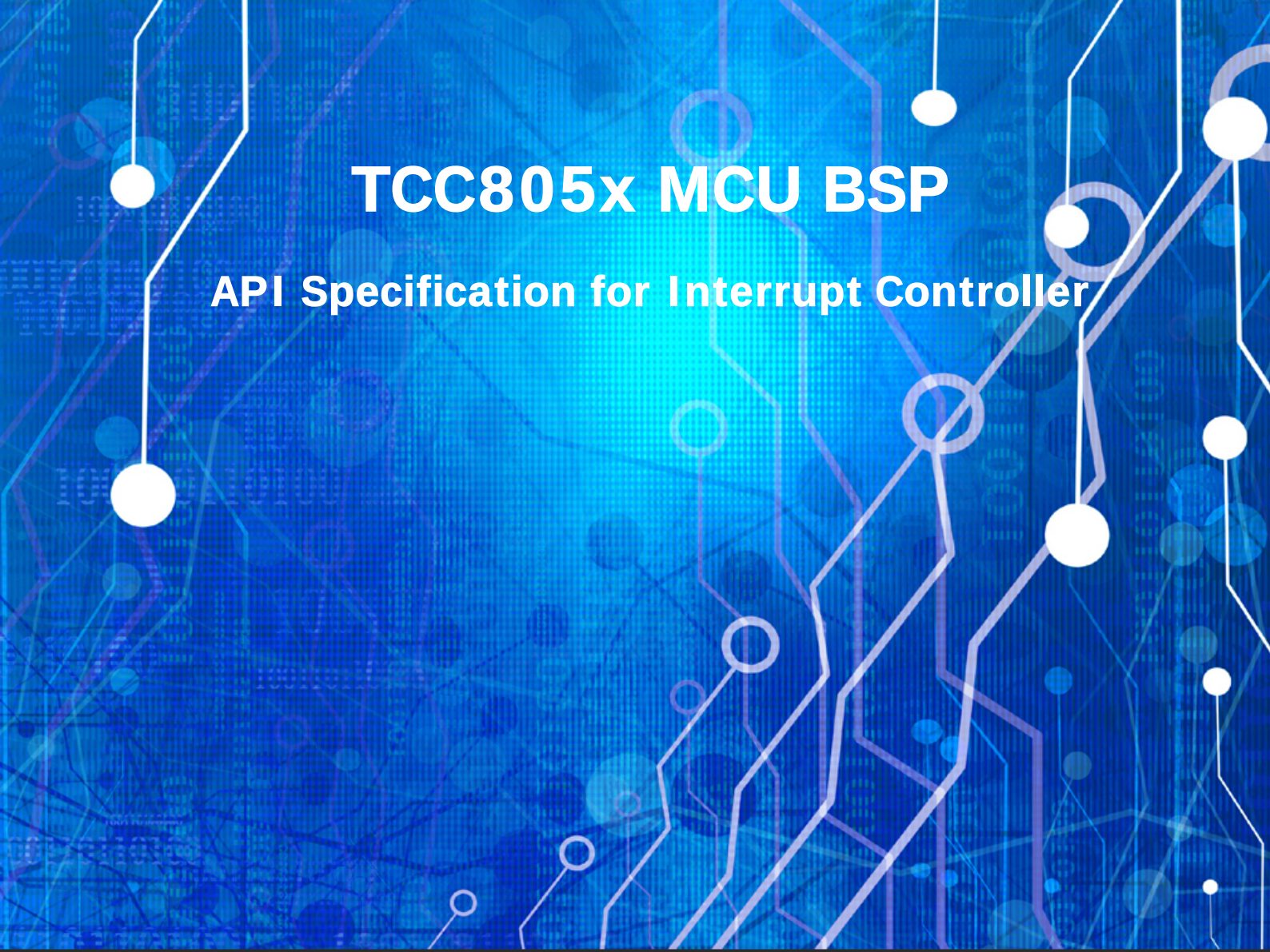




TCC805x MCU BSP

API Specification for Interrupt Controller

Rev. 1.00 [A]

2020-11-12

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit <http://www.telechips.com> for more information.

TABLE OF CONTENTS

Contents

1	Introduction	3
2	Terms and Abbreviations	3
3	Source Files.....	4
3.1	Location of Source Files.....	4
3.2	Simple Description of Source Files	4
4	Enumerations	5
4.1	MCU Shared Peripherals Interrupts.....	5
5	Constants.....	11
5.1	GIC Control Value	11
5.2	Interrupt Type Value	12
5.3	Interrupt Index Value.....	13
6	Structures	14
6.1	GICIntFuncPtr_t	14
6.2	GICDistributor_t	15
6.3	GICCpuInterface_t.....	17
7	Callback Functions	18
7.1	GICIsrFunc.....	18
8	APIs	19
8.1	GIC_Init.....	19
8.2	GIC_IntSrcEn	19
8.3	GIC_IntSrcDis	20
8.4	GIC_IntPrioSet	21
8.5	GIC_IntVectSet.....	22
8.6	GIC_IntExtSet	23
8.7	GIC_IntSGI	24
9	How to use GIC Driver.....	25
9.1	Initialize GIC Driver.....	25
9.2	Example of using GIC in Device Driver.....	25
9.3	Example of using External Interrupt	26
10	References	27
11	Revision History	28
	Rev. 1.00: 2020-11-12	28

Figures

Figure 3.1 Location of Source Files	4
---	---

Tables

Table 2.1 Terms and Abbreviation	3
--	---

1 INTRODUCTION

This document describes the interrupt controller device driver (hereinafter referred to as GIC driver) of TCC805x MCU BSP. For more detailed information of the interrupt controller in the TCC805x MCU Sub-system, refer to Chapter 5 Interrupt Controller of the “*PART 11-MICOM SUB-SYSTEM*” in the “*TCC805x Full Specification*”. [1]

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

GIC	Generic Interrupt Controller
ISR	Interrupt Service Routine

3 SOURCE FILES

3.1 Location of Source Files

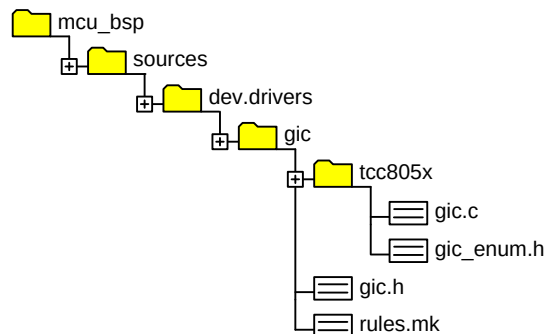


Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- `gic.c`
 - GIC source
- `gic_enum.h`
 - Interrupt Index enumerations
- `gic.h`
 - GIC driver API header
- `rules.mk`
 - Make rules

4 ENUMERATIONS

4.1 MCU Shared Peripherals Interrupts

The Shared Peripherals interrupt index of MCU peripherals

Declaration

```
enum
{
    GIC_CAN0_0      = (GIC_SPI_START + 0UL),
    GIC_CAN0_1      = (GIC_SPI_START + 1UL),
    GIC_CAN1_0      = (GIC_SPI_START + 2UL),
    GIC_CAN1_1      = (GIC_SPI_START + 3UL),
    GIC_CAN2_0      = (GIC_SPI_START + 4UL),
    GIC_CAN2_1      = (GIC_SPI_START + 5UL),
    GIC_GPSB        = (GIC_SPI_START + 6UL),
    GIC_GPSB1       = (GIC_SPI_START + 7UL),
    GIC_GPSB2       = (GIC_SPI_START + 8UL),
    GIC_GPSB3       = (GIC_SPI_START + 9UL),
    GIC_GPSB4       = (GIC_SPI_START + 10UL),
    GIC_GPSB5       = (GIC_SPI_START + 11UL),
    GIC_GPSB0_DMA   = (GIC_SPI_START + 12UL),
    GIC_GPSB1_DMA   = (GIC_SPI_START + 13UL),
    GIC_GPSB2_DMA   = (GIC_SPI_START + 14UL),
    GIC_GPSB3_DMA   = (GIC_SPI_START + 15UL),
    GIC_GPSB4_DMA   = (GIC_SPI_START + 16UL),
    GIC_GPSB5_DMA   = (GIC_SPI_START + 17UL),
    GIC_UART0       = (GIC_SPI_START + 18UL),
    GIC_UART1       = (GIC_SPI_START + 19UL),
    GIC_UART2       = (GIC_SPI_START + 20UL),
    GIC_UART3       = (GIC_SPI_START + 21UL),
    GIC_UART4       = (GIC_SPI_START + 22UL),
    GIC_UART5       = (GIC_SPI_START + 23UL),
    GIC_UT_DMA0     = (GIC_SPI_START + 24UL),
    GIC_UT_DMA1     = (GIC_SPI_START + 25UL),
    GIC_UT_DMA2     = (GIC_SPI_START + 26UL),
    GIC_UT_DMA3     = (GIC_SPI_START + 27UL),
    GIC_UT_DMA4     = (GIC_SPI_START + 28UL),
    GIC_UT_DMA5     = (GIC_SPI_START + 29UL),
    GIC_I2C         = (GIC_SPI_START + 30UL),
    GIC_I2C1        = (GIC_SPI_START + 31UL),
    GIC_I2C2        = (GIC_SPI_START + 32UL),
    GIC_VS_I2C0     = (GIC_SPI_START + 33UL),
    GIC_VS_I2C1     = (GIC_SPI_START + 34UL),
    GIC_VS_I2C2     = (GIC_SPI_START + 35UL),
    GIC_IC_TC0      = (GIC_SPI_START + 36UL),
    GIC_IC_TC1      = (GIC_SPI_START + 37UL),
    GIC_IC_TC2      = (GIC_SPI_START + 38UL),
    GIC_RED_IC_TC0  = (GIC_SPI_START + 39UL),
    GIC_RED_IC_TC1  = (GIC_SPI_START + 40UL),
    GIC_RED_IC_TC2  = (GIC_SPI_START + 41UL),
    GIC_DMA_PL080   = (GIC_SPI_START + 42UL),
    GIC_ADC         = (GIC_SPI_START + 43UL),
    GIC_TIMER_0     = (GIC_SPI_START + 44UL),
```

GIC_TIMER_1	= (GIC_SPI_START + 45UL),
GIC_TIMER_2	= (GIC_SPI_START + 46UL),
GIC_TIMER_3	= (GIC_SPI_START + 47UL),
GIC_TIMER_4	= (GIC_SPI_START + 48UL),
GIC_TIMER_5	= (GIC_SPI_START + 49UL),
GIC_WATCHDOG	= (GIC_SPI_START + 50UL),
GIC_DEFAULT_SLV_ERR	= (GIC_SPI_START + 51UL),
GIC_MBOX0_SLV_S_W	= (GIC_SPI_START + 52UL),
GIC_MBOX0_SLV_S_R	= (GIC_SPI_START + 53UL),
GIC_MBOX0_SLV_NS_W	= (GIC_SPI_START + 54UL),
GIC_MBOX0_SLV_NS_R	= (GIC_SPI_START + 55UL),
GIC_MBOX1_SLV_S_W	= (GIC_SPI_START + 56UL),
GIC_MBOX1_SLV_S_R	= (GIC_SPI_START + 57UL),
GIC_MBOX1_SLV_NS_W	= (GIC_SPI_START + 58UL),
GIC_MBOX1_SLV_NS_R	= (GIC_SPI_START + 59UL),
GIC_MBOX2_SLV_W	= (GIC_SPI_START + 60UL),
GIC_MBOX2_SLV_R	= (GIC_SPI_START + 61UL),
GIC_MBOX0_MST_S_W	= (GIC_SPI_START + 62UL),
GIC_MBOX0_MST_S_R	= (GIC_SPI_START + 63UL),
GIC_MBOX0_MST_NS_W	= (GIC_SPI_START + 64UL),
GIC_MBOX0_MST_NS_R	= (GIC_SPI_START + 65UL),
GIC_MBOX1_MST_S_W	= (GIC_SPI_START + 66UL),
GIC_MBOX1_MST_S_R	= (GIC_SPI_START + 67UL),
GIC_MBOX1_MST_NS_W	= (GIC_SPI_START + 68UL),
GIC_MBOX1_MST_NS_R	= (GIC_SPI_START + 69UL),
GIC_MBOX2_MST_W	= (GIC_SPI_START + 70UL),
GIC_MBOX2_MST_R	= (GIC_SPI_START + 71UL),
GIC_SFMC	= (GIC_SPI_START + 72UL),
GIC_NSAID_FILTER	= (GIC_SPI_START + 73UL),
GIC_SGID_FILTER	= (GIC_SPI_START + 74UL),
GIC_SFMC_MID_FILTER	= (GIC_SPI_START + 75UL),
GIC_IMC_MID_FILTER	= (GIC_SPI_START + 76UL),
GIC_EXT0	= (GIC_SPI_START + 77UL),
GIC_EXT1	= (GIC_SPI_START + 78UL),
GIC_EXT2	= (GIC_SPI_START + 79UL),
GIC_EXT3	= (GIC_SPI_START + 80UL),
GIC_EXT4	= (GIC_SPI_START + 81UL),
GIC_EXT5	= (GIC_SPI_START + 82UL),
GIC_EXT6	= (GIC_SPI_START + 83UL),
GIC_EXT7	= (GIC_SPI_START + 84UL),
GIC_EXT8	= (GIC_SPI_START + 85UL),
GIC_EXT9	= (GIC_SPI_START + 86UL),
GIC_EXTN0	= (GIC_SPI_START + 87UL),
GIC_EXTN1	= (GIC_SPI_START + 88UL),
GIC_EXTN2	= (GIC_SPI_START + 89UL),
GIC_EXTN3	= (GIC_SPI_START + 90UL),
GIC_EXTN4	= (GIC_SPI_START + 91UL),
GIC_EXTN5	= (GIC_SPI_START + 92UL),
GIC_EXTN6	= (GIC_SPI_START + 93UL),
GIC_EXTN7	= (GIC_SPI_START + 94UL),
GIC_EXTN8	= (GIC_SPI_START + 95UL),
GIC_EXTN9	= (GIC_SPI_START + 96UL),
GIC_CR5_PMU	= (GIC_SPI_START + 97UL),
GIC_FMU_IRQ	= (GIC_SPI_START + 98UL),
GIC_FMU_FIQ	= (GIC_SPI_START + 99UL),
GIC_RESERVED_100	= (GIC_SPI_START + 100UL),
GIC_RESERVED_101	= (GIC_SPI_START + 101UL),
GIC_RESERVED_102	= (GIC_SPI_START + 102UL),
GIC_PMGPIO	= (GIC_SPI_START + 103UL),

};

Description

<i>Value</i>	<i>Description</i>
<i>GIC_CAN0_0</i>	<i>CAN FD Controller-0[0]</i>
<i>GIC_CAN0_1</i>	<i>CAN FD Controller-0[1]</i>
<i>GIC_CAN1_0</i>	<i>CAN FD Controller-1[0]</i>
<i>GIC_CAN1_1</i>	<i>CAN FD Controller-1[1]</i>
<i>GIC_CAN2_0</i>	<i>CAN FD Controller-2[0]</i>
<i>GIC_CAN2_1</i>	<i>CAN FD Controller-2[1]</i>
<i>GIC_GPSB</i>	<i>GPSB Controller-0</i>
<i>GIC_GPSB1</i>	<i>GPSB Controller-1</i>
<i>GIC_GPSB2</i>	<i>GPSB Controller-2</i>
<i>GIC_GPSB3</i>	<i>GPSB Controller-3</i>
<i>GIC_GPSB4</i>	<i>GPSB Controller-4</i>
<i>GIC_GPSB5</i>	<i>Reserved</i>
<i>GIC_GPSB0_DMA</i>	<i>GPSB Controller-0 DMA</i>
<i>GIC_GPSB1_DMA</i>	<i>GPSB Controller-1 DMA</i>
<i>GIC_GPSB2_DMA</i>	<i>GPSB Controller-2 DMA</i>
<i>GIC_GPSB3_DMA</i>	<i>GPSB Controller-3 DMA</i>
<i>GIC_GPSB4_DMA</i>	<i>GPSB Controller-4 DMA</i>
<i>GIC_GPSB5_DMA</i>	<i>Reserved</i>
<i>GIC_UART0</i>	<i>UART Controller-0</i>
<i>GIC_UART1</i>	<i>UART Controller-1</i>
<i>GIC_UART2</i>	<i>UART Controller-2</i>
<i>GIC_UART3</i>	<i>UART Controller-3</i>
<i>GIC_UART4</i>	<i>UART Controller-4</i>
<i>GIC_UART5</i>	<i>Reserved</i>
<i>GIC_UT_DMA0</i>	<i>UART DMA Controller-0</i>
<i>GIC_UT_DMA1</i>	<i>UART DMA Controller-1</i>

Value	Description
<i>GIC_UT_DMA2</i>	<i>UART DMA Controller-2</i>
<i>GIC_UT_DMA3</i>	<i>UART DMA Controller-3</i>
<i>GIC_UT_DMA4</i>	<i>UART DMA Controller-4</i>
<i>GIC_UT_DMA5</i>	<i>Reserved</i>
<i>GIC_I2C</i>	<i>I²C Master Controller-0</i>
<i>GIC_I2C1</i>	<i>I²C Master Controller-1</i>
<i>GIC_I2C2</i>	<i>I²C Master Controller-2</i>
<i>GIC_VS_I2C0</i>	<i>I²C Virtual Slave-0</i>
<i>GIC_VS_I2C1</i>	<i>I²C Virtual Slave-1</i>
<i>GIC_VS_I2C2</i>	<i>I²C Virtual Slave-2</i>
<i>GIC_IC_TC0</i>	<i>ICTC-0</i>
<i>GIC_IC_TC1</i>	<i>ICTC-1</i>
<i>GIC_IC_TC2</i>	<i>ICTC-2</i>
<i>GIC_RED_IC_TC0</i>	<i>Redundancy ICTC-0</i>
<i>GIC_RED_IC_TC1</i>	<i>Redundancy ICTC-1</i>
<i>GIC_RED_IC_TC2</i>	<i>Redundancy ICTC-2</i>
<i>GIC_DMA_PL080</i>	<i>General DMA Controller</i>
<i>GIC_ADC</i>	<i>ADC Controller</i>
<i>GIC_TIMER_0</i>	<i>Timer-0</i>
<i>GIC_TIMER_1</i>	<i>Timer-1</i>
<i>GIC_TIMER_2</i>	<i>Timer-2</i>
<i>GIC_TIMER_3</i>	<i>Timer-3</i>
<i>GIC_TIMER_4</i>	<i>Timer-4</i>
<i>GIC_TIMER_5</i>	<i>Timer-5</i>
<i>GIC_WATCHDOG</i>	<i>Windowed Watchdog Timer</i>
<i>GIC_DEFAULT_SLV_ERR</i>	<i>Default Slave Error Interrupt Handler</i>
<i>GIC_MBOX0_SLV_S_W</i>	<i>MICOM Secure Mailbox-0 Terminal, Write</i>
<i>GIC_MBOX0_SLV_S_R</i>	<i>MICOM Secure Mailbox-0 Terminal, Read</i>
<i>GIC_MBOX0_SLV_NS_W</i>	<i>MICOM Non-secure Mailbox-0 Terminal, Write</i>

Value	Description
<i>GIC_MBOX0_SLV_NS_R</i>	<i>MICOM Non-secure Mailbox-0 Terminal, Read</i>
<i>GIC_MBOX1_SLV_S_W</i>	<i>MICOM Secure Mailbox-1 Terminal, Write</i>
<i>GIC_MBOX1_SLV_S_R</i>	<i>MICOM Secure Mailbox-1 Terminal, Read</i>
<i>GIC_MBOX1_SLV_NS_W</i>	<i>MICOM Non-secure Mailbox-1 Terminal, Write</i>
<i>GIC_MBOX1_SLV_NS_R</i>	<i>MICOM Non-secure Mailbox-1 Terminal, Read</i>
<i>GIC_MBOX2_SLV_W</i>	<i>MICOM Mailbox-2 Terminal, Write</i>
<i>GIC_MBOX2_SLV_R</i>	<i>MICOM Mailbox-2 Terminal, Read</i>
<i>GIC_MBOX0_MST_S_W</i>	<i>AP Secure Mailbox-0 Terminal, Write</i>
<i>GIC_MBOX0_MST_S_R</i>	<i>AP Secure Mailbox-0 Terminal, Read</i>
<i>GIC_MBOX0_MST_NS_W</i>	<i>AP Non-secure Mailbox-0 Terminal, Write</i>
<i>GIC_MBOX0_MST_NS_R</i>	<i>AP Non-secure Mailbox-0 Terminal, Read</i>
<i>GIC_MBOX1_MST_S_W</i>	<i>AP Secure Mailbox-1 Terminal, Write</i>
<i>GIC_MBOX1_MST_S_R</i>	<i>AP Secure Mailbox-1 Terminal, Read</i>
<i>GIC_MBOX1_MST_NS_W</i>	<i>AP Non-secure Mailbox-1 Terminal, Write</i>
<i>GIC_MBOX1_MST_NS_R</i>	<i>AP Non-secure Mailbox-1 Terminal, Read</i>
<i>GIC_MBOX2_MST_W</i>	<i>AP Mailbox-2 Terminal, Write</i>
<i>GIC_MBOX2_MST_R</i>	<i>AP Mailbox-2 Terminal, Read</i>
<i>GIC_SFMC</i>	<i>Serial-NOR Flash Memory Controller (SFMC)</i>
<i>GIC_NSAID_FILTER</i>	<i>Non-secure Access ID (NSAID) Filter</i>
<i>GIC_SGID_FILTER</i>	<i>Secure Group ID (SGID) Filter</i>
<i>GIC_SFMC_MID_FILTER</i>	<i>Serial-NOR Flash Memory Controller (SFMC) MID Filter</i>
<i>GIC_IMC_MID_FILTER</i>	<i>Internal Memory Controller (IMC) MID Filter</i>
<i>GIC_EXT0</i>	<i>External IRQ input-0</i>
<i>GIC_EXT1</i>	<i>External IRQ input-1</i>
<i>GIC_EXT2</i>	<i>External IRQ input-2</i>
<i>GIC_EXT3</i>	<i>External IRQ input-3</i>
<i>GIC_EXT4</i>	<i>External IRQ input-4</i>
<i>GIC_EXT5</i>	<i>External IRQ input-5</i>
<i>GIC_EXT6</i>	<i>External IRQ input-6</i>

Value	Description
<i>GIC_EXT7</i>	<i>External IRQ input-7</i>
<i>GIC_EXT8</i>	<i>External IRQ input-8</i>
<i>GIC_EXT9</i>	<i>External IRQ input-9</i>
<i>GIC_EXTn0</i>	<i>External IRQn input-0</i>
<i>GIC_EXTn1</i>	<i>External IRQn input-1</i>
<i>GIC_EXTn2</i>	<i>External IRQn input-2</i>
<i>GIC_EXTn3</i>	<i>External IRQn input-3</i>
<i>GIC_EXTn4</i>	<i>External IRQn input-4</i>
<i>GIC_EXTn5</i>	<i>External IRQn input-5</i>
<i>GIC_EXTn6</i>	<i>External IRQn input-6</i>
<i>GIC_EXTn7</i>	<i>External IRQn input-7</i>
<i>GIC_EXTn8</i>	<i>External IRQn input-8</i>
<i>GIC_EXTn9</i>	<i>External IRQn input-9</i>
<i>GIC_CR5_PMU</i>	<i>Cortex-R5 Performance Monitoring Unit (PMU)</i>
<i>GIC_FMU_IRQ</i>	<i>Fault Management Unit (FMU) IRQ</i>
<i>GIC_FMU_FIQ</i>	<i>Fault Management Unit (FMU) FIQ</i>
<i>GIC_RESERVED_100</i>	<i>Reserved</i>
<i>GIC_RESERVED_101</i>	<i>Reserved</i>
<i>GIC_RESERVED_102</i>	<i>Reserved</i>
<i>GIC_PMGPIO</i>	<i>Power Management I/O (PMIO)</i>

Remark

None

5 CONSTANTS

5.1 GIC Control Value

These constants are used to configure 'GIC's distributor' and 'CPU interface'.

Declaration

```
#define ARM_BIT_GIC_DIST_ICDDCR_EN    (0x00000001UL)
#define GIC_CPUIF_CTRL_ENABLEGRP0    (0x00000001UL)
#define GIC_CPUIF_CTRL_ENABLEGRP1    (0x00000002UL) //Currently not used
#define GIC_CPUIF_CTRL_ACKCTL        (0x00000004UL) //Currently not used
#define GIC_SGI_TO_TARGETLIST        (0UL)
```

Description

Value	Description
ARM_BIT_GIC_DIST_ICDDCR_EN	Global enable for forwarding pending Group 0 interrupts from the Distributor to the CPU interfaces.
GIC_CPUIF_CTRL_ENABLEGRP0	Enable for the signaling of Group 0 interrupts by the CPU interface to the connected processor.
GIC_CPUIF_CTRL_ENABLEGRP1	Enable for the signaling of Group 1 interrupts by the CPU interface to the connected processor (Currently <i>not</i> used) If you enable this function, you have to implement the relevant code.
GIC_CPUIF_CTRL_ACKCTL	Currently not used. ARM deprecates use of GICC_CTLR.AckCtl , and strongly recommends using a software model where GICC_CTLR.AckCtl is set to 0.
GIC_SGI_TO_TARGETLIST	Forward the SGI to the CPU interfaces specified in the CPUPendingList field of GICD_SGIR. In GIC_IntSGI() function, CPUPendingList field is set to CPU #0 because R5 is single core.

Remark

None

5.2 Interrupt Type Value

These constants are used to determine the type of interrupt trigger.

Declaration

```
#define GIC_INT_TYPE_LEVEL_HIGH      (0x1UL)
#define GIC_INT_TYPE_LEVEL_LOW      (0x2UL)
#define GIC_INT_TYPE_EDGE_RISING    (0x4UL)
#define GIC_INT_TYPE_EDGE_FALLING   (0x8UL)
#define GIC_INT_TYPE_EDGE_BOTH      (GIC_INT_TYPE_EDGE_RISING | GIC_INT_TYPE_EDGE_FALLING)
```

Description

Value	Description
<i>GIC_INT_TYPE_LEVEL_HIGH</i>	<i>Level - sensitive</i>
<i>GIC_INT_TYPE_LEVEL_LOW</i>	<i>Level - sensitive (unused define)</i>
<i>GIC_INT_TYPE_EDGE_RISING</i>	<i>edge-triggered</i>
<i>GIC_INT_TYPE_EDGE_FALLING</i>	<i>edge-triggered (unused define)</i>
<i>GIC_INT_TYPE_EDGE_BOTH</i>	<i>Only supported in external interrupt (GIC_EXT0 ~ GIC_EXT9). Refer to 4.1.</i>

Remark

None

5.3 Interrupt Index Value

These constants are used for reference in the GIC driver and the device drivers.

Declaration

```
#define GIC_PPI_START          (16UL)
#define GIC_SPI_START          (32UL)

#define GIC_EINT_START_INT      (GIC_EXT0)
#define GIC_EINT_END_INT        (GIC_EXT9)
#define GIC_EINT_NUM            (GIC_EXTn0 - GIC_EXT0)

#define GIC_SMU_CB_WDT0         (GIC_SPI_START + 286UL)
#define GIC_SMU_PMU_WDT         (GIC_SPI_START + 294UL)
#define GIC_SMU_HSMB_MBOX3_M    (GIC_SPI_START + 414UL)
#define GIC_SMU_HSMB_MBOX3_M_TXDONE (GIC_SPI_START + 442UL)
#define GIC_INT_SRC_CNT         (GIC_SPI_START + 480UL)
```

Description

Value	Description
<i>GIC_PPI_START</i>	<i>Private Peripherals Interrupt start index</i>
<i>GIC_SPI_START</i>	<i>Shared Peripherals Interrupt start index</i>
<i>GIC_EINT_START_INT</i>	<i>External Interrupt start index</i>
<i>GIC_EINT_END_INT</i>	<i>External Interrupt end index</i>
<i>GIC_EINT_NUM</i>	<i>External Interrupt total number</i>
<i>GIC_SMU_CB_WDT0</i>	<i>CB WDT0 interrupt index</i>
<i>GIC_SMU_PMU_WDT</i>	<i>PMU (Power Management Unit) WDT interrupt index</i>
<i>GIC_SMU_HSMB_MBOX3_M</i>	<i>HSMB_MBOX3_M interrupt index</i>
<i>GIC_SMU_HSMB_MBOX3_M_TXDONE</i>	<i>HSMB_MBOX3_M_TXDONE interrupt index</i>
<i>GIC_INT_SRC_CNT</i>	<i>Total count of Interrupt source</i>

Remark

None

6 STRUCTURES

6.1 GICIntFuncPtr_t

GIC Driver manages ISR callback function pointer for each interrupt index, and *GICIntFuncPtr_t* is a structure for the GIC's management of ISR callback function pointer.

Declaration

```
typedef struct _GICIntFuncPtr_  
{  
    GICIsrFunc ifpFunc;  
    uint8      ifpIsBothEdge;  
    void *     ifpArg;  
} GICIntFuncPtr_t;
```

Description

Value	Description
ifpFunc	Callback function for ISR. Refer to the GICIsrFunc .
ifpIsBothEdge	If this value is true, it means that ifpFunc(ISR callback) is called on rising and falling edges of interrupt source. Both edges are only supported by external interrupt (GIC_EXT0 ~ GIC_EXT9).
ifpArg	ifpFunc input argument

Remark

None

6.2 GICDistributor_t

The GIC's Distributor register is handled using the following structure.

Declaration

```
typedef struct _GICDistributor_  
{  
    uint32 dCTRL;  
    uint32 dTYPER;  
    uint32 dIIDR;  
    uint32 dRSVD1[29];  
    uint32 dIGROUPRn[32];  
    uint32 dISENABLERn[32];  
    uint32 dICENABLERn[32];  
    uint32 dISPENDRn[32];  
    uint32 dICPENDRn[32];  
    uint32 dISACTIVERn[32];  
    uint32 dICACTIVERn[32];  
    uint32 dIPRIORITYRn[255];  
    uint32 dRSVD3[1];  
    uint32 dITARGETSRn[255];  
    uint32 dRSVD4[1];  
    uint32 dICFGRn[64];  
    uint32 dPPISR[1];  
    uint32 dSPISRn[15];  
    uint32 dRSVD5[112];  
    uint32 dSGIR;  
    uint32 dRSVD6[3];  
    uint32 dCPENDSGIRn[4];  
    uint32 dSPENDSGIRn[4];  
} GICDistributor_t;
```


Description

Value	Description
<i>dCTRL</i>	<i>GICD_CTRL, Distributor Control Register</i>
<i>dTYPER</i>	<i>GICD_TYPER, Interrupt Controller Type Register</i>
<i>dIIDR</i>	<i>GICD_IIDR, Distributor Implementer Identification Register</i>
<i>dIGROUPRn</i>	<i>GICD_IGROUPRn, Interrupt Group Registers</i>
<i>dISENABLERn</i>	<i>GICD_ISENABLERn, Interrupt Set-Enable Registers</i>
<i>dICENABLERn</i>	<i>GICD_ICENABLERn, Interrupt Clear-Enable Registers</i>
<i>dISPENDRn</i>	<i>GICD_ISPENDRn, Interrupt Set-Pending Registers</i>
<i>dICPENDRn</i>	<i>GICD_ICPENDRn, Interrupt Clear-Pending Registers</i>
<i>dISACTIVERn</i>	<i>GICD_ISACTIVERn, Interrupt Set-Active Registers</i>
<i>dICACTIVERn</i>	<i>GICD_ICACTIVERn, Interrupt Clear-Active Registers</i>
<i>dIPRIORITYRn</i>	<i>GICD_IPRIORITYRn, Interrupt Priority Registers</i>
<i>dITARGETSRn</i>	<i>GICD_ITARGETSRn, Interrupt Processor Targets Registers</i>
<i>dICFGRn</i>	<i>GICD_ICFGRn, RW IMPLEMENTATION DEFINED Interrupt Configuration Registers</i>
<i>dPPISR</i>	<i>GICD_PPISR, Private Peripheral Interrupt Status Register</i>
<i>dSPISRn</i>	<i>GICD_SPISRn, Shared Peripheral Interrupt Status Registers</i>
<i>dSGIR</i>	<i>GICD_SGIR, Software Generate Interrupt Register</i>
<i>dCPENDSGIRn</i>	<i>GICD_CPENDSGIRn, SGInterrupt Clear-Active Registers</i>
<i>dSPENDSGIRn</i>	<i>GICD_SPENDSGIRn, SGInterrupt Set-Active Registers</i>

Remark

None

6.3 GICCpuInterface_t

The GIC's CPU-interface register is handled using the following structure.

Declaration

```
typedef struct _GICCpuInterface_
{
    uint32 cCTRL;
    uint32 cPMR;
    uint32 cBPR;
    uint32 cIAR;
    uint32 cEOIR;
    uint32 cRPR;
    uint32 cHPPIR;
    uint32 cABPR;
    uint32 cAIAR;
    uint32 cAEOIR;
    uint32 cAHPPIR;
    uint32 cRSVD[52];
    uint32 cIIDR;
} GICCpuInterface_t;
```

Description

Value	Description
cCTRL	GICC_CTRL, CPU Interface Control Register
cPMR	GICC_PMR, Interrupt Priority Mask Register
cBPR	GICC_BPR, Binary Point Register
cIAR	GICC_IAR, Interrupt Acknowledge Register
cEOIR	GICC_EOIR, End Interrupt Register
cRPR	GICC_RPR, Running Priority Register
cHPPIR	GICC_HPPIR, Highest Pending Interrupt Register
cABPR	GICC_ABPR, Aliased Binary Point Register
cAIAR	GICC_AIAR, Aliased Interrupt Acknowledge Register
cAEOIR	GICC_AEOIR, Aliased End Interrupt Register
cAHPPIR	GICC_AHPPIR, Aliased Highest Pending Interrupt Register
cIIDR	GICC_IIDR, CPU Interface Identification Register

Remark

None

7 CALLBACK FUNCTIONS

7.1 GICIsrFunc

This is the 4th parameter type of GIC_IntVectSet. When an interrupt registered by GIC_IntVectSet occurs, the registered callback function corresponding to the interrupt is called.

Declaration

```
typedef void (*GICIsrFunc)
(
    void * pArg
);
```

Parameters

Value	Description
pArg	[in/out] callback function argument

Return

None

Remark

None

8 APIs

8.1 GIC_Init

Initializes the GIC400 block.

Declaration

```
void GIC_Init  
(  
    void  
);
```

Parameters

None

Return

None

Remark

None

8.2 GIC_IntSrcEn

Enables interrupt source id.

Declaration

```
SALRetCode_t GIC_IntSrcEn  
(  
    uint32 uiIntId  
);
```

Parameters

Value	Description
uiIntId	[in] an index of interrupt source (range of param: 0 ~ GIC_INT_SRC_CNT)

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

8.3 GIC_IntSrcDis

Disables interrupt source id.

Declaration

```
SALRetCode_t GIC_IntSrcDis
(
    uint32 uiIntId
);
```

Parameters

Value	Description
<i>uiIntId</i>	<i>[in] an index of interrupt source (range of param: 0 ~ GIC_INT_SRC_CNT)</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

8.4 GIC_IntPrioSet

Sets priority of interrupt source id.

Declaration

```
SALRetCode_t GIC_IntPrioSet  
(  
    uint32 uiIntId,  
    uint32 uiPrio  
);
```

Parameters

Value	Description
<i>uiIntId</i>	<i>[in] an index of interrupt source (range of param: 0 ~ GIC_INT_SRC_CNT)</i>
<i>uiPrio</i>	<i>[in] priority (range of param: 0~15)</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

8.5 GIC_IntVectSet

Sets priority of interrupt source id, interrupt type, ISR callback function pointer

Declaration

```
SALRetCode_t GIC_IntVectSet
(
    uint32    uiIntId,
    uint32    uiPrio,
    uint8     ucIntType,
    GICIsrFunc fnIntFunc,
    void*     pIntArg
);
```

Parameters

Value	Description
<i>uiIntId</i>	<i>[in] an index of interrupt source (range of param: 0 ~ GIC_INT_SRC_CNT)</i>
<i>uiPrio</i>	<i>[in] priority (range of param: 0~15)</i>
<i>ucIntType</i>	<i>[in] interrupt type (range of param: refer to 5.2 Interrupt Type Value)</i>
<i>fnIntFunc</i>	<i>[in] ISR Callback function pointer. Refer to the GICIsrFunc.</i>
<i>pIntArg</i>	<i>[in] ISR Callback function parameter pointer</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

8.6 GIC_IntExtSet

Sets external interrupt.

Declaration

```
SALRetCode_t GIC_IntExtSet
(
    uint32 uiIntId,
    uint32 uiGpio
);
```

Parameters

Value	Description
<i>uiIntId</i>	<i>[in]</i> an index of interrupt source (range of param: GIC_EXT0 ~ GIC_EXT9)
<i>uiGpio</i>	<i>[in]</i> GPIO port (refer to GPIO_GPA(x), GPIO_GPB(x), GPIO_GPC(x), TCC_GPD(x), GPIO_GPE(x), TCC_GPF(x), GPIO_GPG(x), GPIO_GPH(x), GPIO_GPK(x), GPIO_GPMA(x), GPIO_GPMB(x), GPIO_GPMC(x), GPIO_GPMD(x) macro define of gpio.h)

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

8.7 GIC_IntSGI

Generates SGI

Declaration

```
SALRetCode_t GIC_IntSGI
(
    uint32 uiIntId
);
```

Parameters

Value	Description
uiIntId	[in] an index of interrupt source (range of param : 0 ~ GIC_INT_SRC_CNT)

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

9 HOW TO USE GIC DRIVER

9.1 Initialize GIC Driver

```
void BSP_PreInit(void)
{
    CLOCK_Init();
    GIC_Init();
}
```

9.2 Example of using GIC in Device Driver

The following example shows how to use GIC in device driver on GPSB device driver.

```
void GPSB_Init(void)
{
    SALRetCode_t err = SAL_RET_SUCCESS;

    /* Register gpsb irq handler */
    err = GIC_IntVectSet((uint32)GIC_GPSB,
                        (uint32)GIC_PRIORITY_NO_MEAN,
                        (uint8)GIC_INT_TYPE_LEVEL_HIGH,
                        &GPSB_Isr0, NULL_PTR);

    if(err != SAL_RET_SUCCESS)
    {
        GPSB_D("%s failed to set interrupt vector %d\n", __func__, GIC_GPSB);
    }
    GIC_IntSrcEn(GIC_GPSB);
}
```

9.3 Example of using External Interrupt

The following is an example code that uses an external interrupt, and GPIOA00 is used as the door open signal input.

```
static uint32 gIsDoorDet = 0UL;

static void VehicleSignalExternIntDoor(void *pArg)
{
    uint8 ucDet;

    (void)pArg;
    ucDet = GPIO_Get(GPIO_GPA(0UL));
    gIsDoorDet = (ucDet != 0UL) ? (1UL) : (0UL);
}

static void VehicleDoorOpen(void)
{
    // DOOR Open
    (void) GPIO_Config(GPIO_GPA(0UL), (GPIO_FUNC(0) | GPIO_INPUT|GPIO_INPUTBUF_EN));
    (void)GIC_IntVectSet(GIC_EXT3, GIC_PRIORITY_NO_MEAN,
                        GIC_INT_TYPE_EDGE_BOTH,
                        VehicleSignalExternIntDoor,
                        NULL_PTR);

    (void)GIC_IntSrcEn(GIC_EXT3);
    (void)GIC_IntExtSet(GIC_EXT3, GPIO_GPA(0UL));
}
```

10 REFERENCES

- [1] TCC805x Full Specification
- [2] Contact Telechips for more details: sales@telechips.com

11 REVISION HISTORY

Rev. 1.00: 2020-11-12

- First version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."

"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."